REPUBLIQUE DU CAMEROUN
*****
Paix-Travail-Patrie
*****
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
*****
Université de Yaoundé 1
*****
Institut Saint Jean

REPUBLIC OF CAMEROON
*****
Peace-Work-Fatherland
*****
MINISTRY OF HIGHER EDUCATION
*****
University of Yaounde 1
*****
Saint Jean Institute

INSTITUT UNIVERSITAIRE SAINT JEAN
SAINT JEAN INGÉNIEUR

PROJET SYSTEME D'EXPLOITATION

Conception et mise en œuvre de File Analyzer, un analyseur avancé de fichiers : cas de EXT4 sous Debian

MEMBRE DU GROUPE :

- SONGMENE LADO Belviane

Sous la direction de:

- M. NGUIMBUS Emmanuel

ANNEE ACADEMIQUE: 2025-2026

SOMMAIRE

SOMMAIRE i
LISTE DES TABLEAUX iii
LISTE DES FIGURES iv
PARTIE 1 : CAHIER DES CHARGES 1
INTRODUCTION 2
I. CONTEXTE ET JUSTIFICATION DU PROJET 3
II. PROLEMATIQUE 4
III. LES OBJECTIFS DU PROJET 4
IV. EXPRESSION DES BESOINS 5
V. ACTEURS DU PROJET 8
VI. ESTIMATION DU COUT DU PROJET 8
VII. PLANIFICATION DU PROJET 10
VIII. LES CONTRAINTES DU PROJET 11
IX. LES LIVRABLES 11
CONCLUSION 12
PARTIE 2: CAHIER D'ANALYSE 13
INTRODUCTION 14
I. CONTEXTE ET OBJECTIF 15
II. PERIMETRE FONCTIONNEL 15
III. MODELISATION UML 16
IV. LIVRABLES 27
CONCLUSION 28
PARTIE 3: CAHIER DE CONCEPTION 29
INTRODUCTION 30
I. DIAGRAMMES DE SEQUENCE 31
II. DIAGRAMMES DE CLASSE 33
III. LIVRABLES 33
CONCLUSION 34
PARTIE 4: CAHIER DE REALISATION 35
Introduction 36

I. Architecture générale 37
II. Conception détaillée 38
III. Déroulement de la réalisation 38
IV. Tests réalisés 39
PARTIE 5: CAHIER DE DEPLOIEMENT 40
I. INTRODUCTION 41
II. PRÉREQUIS SYSTÈME 41
III. MÉTHODES D'INSTALLATION 42
IV. CRÉATION DU PACKAGE .DEB 44
V. DÉPLOIEMENT 48
VI. CONFIGURATION 49
VII. DÉPANNAGE 49
VIII. DÉSINSTALLATION 50
CONCLUSION 51
TABLES DES MATIERES 52

LISTE DES TABLEAUX

Tableau 1: Acteurs du projet 8
Tableau 2: Ressources Matérielles 8
Tableau 3: Ressources Logicielles 9
Tableau 4: Ressources humaines 9
Tableau 5: Tableau des taches 10
Tableau 6: CU1-Selectionner un fichier 18
Tableau 7: CU2-Analyser un fichier 19
Tableau 8: CU3-Visualiser les résultats 20
Tableau 9: CU4-Exporter le rapport d'analyse 21
Tableau 10: CU5-Changer de vue d'affichage 22
Tableau 11: CU6-Accéder à l'aide contextuelle 23
Tableau 12:CU7-Configurer les paramètres d'analyse 24
Tableau 13: CU8-Analyser plusieurs fichiers séquentiellement 25
Tableau 14: CU-Visualiser la carte physique des blocs 26
Tableau 15: Configuration matérielle minimale 41
Tableau 16: Configuration logicielle 41
Tableau 17: Erreurs courantes 49

LISTE DES FIGURES

Figure 1: Diagramme de Gantt 10
Figure 2: Diagramme des cas d'utilisation 17
Figure 3: séquence analyser un fichier 31
Figure 4: Séquence visualiser la carte physique des blocs 32
Figure 5: Diagramme de classe 33

PARTIE 1 : CAHIER DES CHARGES

INTRODUCTION

Le cahier des charges est un document de référence qui présente, de manière structurée et synthétique, l'ensemble des fonctions que la solution finale devra offrir. Il décrit les besoins à satisfaire, les spécifications techniques auxquelles le projet devra se conformer, ainsi que les contraintes à respecter. Élaboré conjointement entre le commanditaire et le commandité, ce document contractuel sert de base commune pour orienter, encadrer et vérifier la réalisation du projet. Dans cette perspective, le présent cahier des charges expose le contexte dans lequel s'inscrit le projet, les objectifs visés, les besoins exprimés, les acteurs concernés et les différentes contraintes identifiées. Il précise également la planification générale, les livrables attendus ainsi que les critères d'évaluation qui permettront de mesurer la conformité de la solution développée. Ce document constitue ainsi un outil essentiel pour assurer le bon déroulement du projet et garantir que la solution proposée répondra fidèlement aux attentes définies en amont.

I. CONTEXTE ET JUSTIFICATION DU PROJET

1. Contexte

Les systèmes de fichiers existent depuis plusieurs décennies. Dès les premières technologies de stockage, comme les cartes perforées ou les bandes magnétiques, ils permettaient déjà d'organiser les données, bien que de manière limitée. Ces anciens supports ne proposaient qu'un accès linéaire, nécessitant de parcourir séquentiellement l'ensemble du support pour atteindre une information précise, ce qui entraînait des temps d'accès importants.

Avec l'évolution des technologies de stockage, les systèmes de fichiers modernes offrent désormais un accès aléatoire, permettant de récupérer ou de modifier rapidement n'importe quelle donnée à partir de son emplacement. Un système de fichiers constitue ainsi un composant essentiel d'un système d'exploitation : il assure l'organisation, le stockage, la gestion et l'accès aux données sur un support tel qu'un disque dur, une clé USB ou une mémoire flash. Il joue également un rôle d'interface entre les applications et le matériel, rendant possibles des opérations comme la création, la lecture, l'écriture ou la suppression de fichiers.

Parmi les systèmes de fichiers largement utilisés dans les environnements Linux, **EXT4 (*Extended filesystem-version4*)** occupe une place de référence grâce à sa robustesse, ses performances et ses fonctionnalités avancées, notamment la gestion des extents, la journalisation et la réduction de la fragmentation.

2. Justification

Dans ce contexte, l'analyse du fonctionnement interne d'EXT4 reste une démarche complexe, souvent réservée aux outils spécialisés ou à des utilisateurs expérimentés. Les utilitaires standards offrent une vision partielle, tandis que les outils avancés sont parfois difficiles à interpréter.

Le projet propose donc la conception et la mise en œuvre d'un outil nommé **File Analyzer**, destiné à simplifier et à rendre accessible l'analyse détaillée des fichiers stockés sur une partition EXT4 sous Debian. Cet outil permettra d'extraire, d'interpréter et de présenter de manière claire l'ensemble des informations techniques et structurées relatives à un fichier : métadonnées, fragmentation, type de fichier, taille, permissions, et autres éléments internes.

II. PROLEMATIQUE

Malgré l'adoption généralisée d'EXT4 dans les environnements Linux, l'analyse approfondie de la structure interne des fichiers demeure complexe et fragmentée. Les utilisateurs sont confrontés à plusieurs obstacles :

- Les utilitaires standards (ls, stat, file) n'exposent qu'une vue superficielle des métadonnées, sans accéder aux structures internes d'EXT4 (inodes, extents, superblocks) ;
- L'analyse complète nécessite la combinaison de multiples outils spécialisés (debugfs, dumpe2fs, filefrag, xxd) dont l'utilisation requiert une expertise approfondie ;
- Aucune solution unifiée ne permet de corréler simultanément le type du fichier, ses métadonnées système, son organisation physique sur le disque et son état de fragmentation ;
- L'absence d'interface intuitive limite la compréhension concrète du fonctionnement interne d'EXT4, pourtant essentiel dans la formation en systèmes d'exploitation.

Question principale :

Comment concevoir un outil unique et accessible permettant d'analyser en profondeur la structure complète d'un fichier sur une partition EXT4 sous Debian, en exposant de manière intégrée ses métadonnées, son organisation physique et son état de fragmentation ?

III. LES OBJECTIFS DU PROJET

Les objectifs définissent la finalité de notre projet et se formulent en verbe à l'infinitif. Il est suffisamment détaillé pour permettre la planification et l'évaluation des activités de notre projet.

1. Objectif global

L'objectif principal, qui est le pourquoi de notre projet est **de Faciliter la compréhension et l'analyse approfondie des systèmes de fichiers EXT4** en fournissant un outil unifié permettant d'explorer de manière claire et accessible la structure interne, les métadonnées et l'organisation physique des fichiers sous Debian.

2. Objectifs spécifiques

Les axes de travail pour atteindre l'objectif global sont :

- Identifier automatiquement le type du fichier analysé et extraire l'ensemble des champs de son header (magic number, sections, métadonnées structurelles) ;

- Extraire et présenter les métadonnées système du fichier : permissions, propriétaire/groupe, taille logique, taille sur disque, horodatages (atime, mtime, ctime, btime), nombre de liens physiques et flags système (immutable, append-only, etc.) ;
- Cartographier l'organisation physique du fichier sur le disque en listant ses extents avec leurs caractéristiques (offset logique, longueur, numéros de blocs physiques) ;
- Évaluer le niveau de fragmentation du fichier en calculant son taux de fragmentation à partir de la distribution de ses blocs sur le disque ;
- Afficher le contenu brut du fichier lorsqu'il s'agit d'un fichier texte ou lisible, tout en indiquant clairement les fichiers binaires non affichables.

IV. EXPRESSION DES BESOINS

Lorsque nous parlons de besoins ici, il s'agit des différentes fonctionnalités liées au projet. Il existe ainsi des **besoins fonctionnels et des besoins non fonctionnels**.

1. Besoins Fonctionnels

Les besoins fonctionnels représentent l'ensemble des actions que File Analyzer devra permettre d'effectuer.

i. Module « Détection et analyse du type de fichier »

Ce module constitue la première étape de l'analyse avancée. Il permet de :

- Détecter automatiquement le type du fichier analysé (texte, binaire, exécutable, image, PDF, etc.) ;
- Extraire et afficher tous les champs du header du fichier (magic number, sections, métadonnées structurelles).

ii. Module « Extraction des métadonnées système »

Ce module permet d'accéder aux informations système associées au fichier. Il est chargé de :

- Extraire les métadonnées stockées dans l'inode du fichier (permissions, propriétaire, groupe, taille logique et physique) ;
- Récupérer les horodatages complets (atime, mtime, ctime, btime), le nombre de liens physiques et les flags système (immutable, append-only, etc.).

iii. Module « Analyse de l'organisation physique »

Ce module cartographie la disposition réelle du fichier sur le disque. Il est chargé de :

- Identifier et lister l'ensemble des extents utilisés par le fichier ;
- Extraire pour chaque extent son offset logique, sa longueur en blocs et les numéros de blocs physiques correspondants sur le disque.

iv. Module « Calcul de fragmentation »

Ce module évalue le niveau de dispersion du fichier sur le support de stockage. Il est chargé de :

- Calculer le taux de fragmentation en analysant le nombre d'extents et leur distribution sur le disque ;
- Fournir une interprétation qualitative du niveau de fragmentation (faible, modéré, élevé).
- Afficher le taux final sous forme de pourcentage.

v. Module « Affichage du contenu »

Ce module permet de visualiser le contenu interne du fichier analysé. Il est chargé de :

- Afficher le contenu brut des fichiers texte ou lisibles de manière formatée ;
- De visualiser les informations analysées sous forme de panneaux ou onglets.
- Détecter les fichiers binaires et proposer un affichage hexadécimal ou signaler leur nature non affichable directement.

vi. Module « Interface utilisateur graphique (GUI) »

Ce module constitue le point d'interaction entre l'utilisateur et l'outil d'analyse. Il est chargé de :

- Fournir une interface graphique intuitive permettant la sélection du fichier à analyser ;
- Organiser et présenter les résultats d'analyse de manière structurée (onglets, sections) et permettre leur exportation (texte, PDF, HTML).

vii. Module « Gestion des erreurs et logs »

Ce module assure la robustesse et la traçabilité de l'application. Il est chargé de :

- Capturer et gérer les exceptions liées à l'accès aux fichiers ou aux structures EXT4 ;
- Afficher des messages d'erreur explicites à l'utilisateur et enregistrer les logs d'utilisation pour faciliter le débogage.

2. Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes et les qualités que doit respecter l'outil.

i. Performance

L'outil doit garantir des temps de réponse rapides, avec une analyse de fichier de taille moyenne (< 100 Mo) effectuée en moins de 5 secondes, tout en maintenant une interface réactive même pour les fichiers volumineux.

ii. Compatibilité

L'outil sera développé en Python et fonctionnera exclusivement sur Debian 13.1.0 (Trixie) avec support du système de fichiers EXT4, nécessitant des privilèges root pour accéder aux structures bas niveau.

iii. Fiabilité

L'outil doit fonctionner en mode lecture seule sans jamais modifier ni corrompre les fichiers analysés, tout en gérant de manière robuste les erreurs d'accès, les permissions insuffisantes et les structures de fichiers corrompues.

iv. Utilisabilité

L'interface graphique doit être intuitive et accessible aux utilisateurs sans expertise technique avancée, avec des messages d'erreur clairs, une documentation complète et une aide contextuelle intégrée.

v. Maintenabilité

Le code Python doit suivre les conventions PEP 8, être structuré en modules indépendants, correctement commenté et documenté pour faciliter les évolutions futures et l'ajout de nouvelles fonctionnalités.

vi. Portabilité

L'application sera développée en Python 3.x en utilisant exclusivement des bibliothèques disponibles dans les dépôts officiels Debian pour garantir une installation et un déploiement simplifiés sans dépendances externes complexes.

V. ACTEURS DU PROJET

Il sera question ici de présenter les acteurs du projet :

Tableau 1: Acteurs du projet

Dénomination	Fonction	Statuts	Rôles
M. NGUIMBUS Emmanuel	Maitre d'ouvrage	Enseignant	Définir les besoins du projet Orientation, supervision du projet
SONGMENE LADO Belviane	Maitre d'œuvre	Etudiante	Réalisation et mise en œuvre du projet

VI. ESTIMATION DU COUT DU PROJET

Pour la réalisation de ce projet nous devons avoir à notre disposition des ressources matérielles, logicielles et humaines.

1. Ressources matérielles

Tableau 2: Ressources Matérielles

Matériels	Quantité	Utilité	Prix Unitaire (Fcfa)	Prix Total (Fcfa)
Disques durs 1Tb	1	-Utiliser pour les sauvegardes du projet.	40 000	40 000
Laptop Core i5 5iem generation minimum 8Gb RAM 300GO SSD	2	-Utiliser pour la conception et la mise en œuvre de File Analyzer	300 000	600 000
Clé USB 64Go	2	Utiliser pour le partage de fichiers	5000	10 000
Serveur de développement	1	Serveur pour tester et exécuter l'analyseur de fichiers	250 000	250 000

Connexion Internet	2	Accès à internet pour télécharger des outils de développement et mettre à jour les systèmes.	30 000	60 000
Total				**960 000**

2. Ressources logicielles

Tableau 3: Ressources Logicielles

Logiciels	Description	Prix Unitaire (Fcfa)	Prix Total (Fcfa)
Système d'exploitation Debian 13.1.0	Debian est un système libre et gratuit.	0	0
VirtualBox	Logiciel de virtualisation	0	0
Git	Outil de gestion de version.	0	0
Total			**0**

3. Ressources humaines

La réalisation d'un tel système nécessite l'intervention d'un analyste concepteur pour penser comment le système doit fonctionner, présenter les fonctionnalités possibles à développer.

Tableau 4: Ressources humaines

Rôle	Nombre	Durée de travail en jours	Montant (par jour) en Fcfa	Montant Total (Fcfa)
Analyste concepteur	1	20	80 000	1 600 000
Chef de projet (suivi continu)	1	55	90 000	4 950 000
Développeur	2	35	62 000	2 170 000
Total	**4**	**55**		**8 720 000**

VII. PLANIFICATION DU PROJET

La **planification d'un projet** est l'activité qui consiste à ordonnancer les tâches à réaliser, qui ont été formalisées lors de la structuration du projet, estimer les charges associées, déterminer les profils nécessaires à leur réalisation. Tout activités étant toujours limitée dans le temps, il est préférable de planifier notre travail afin d'éviter les pertes de temps. Dans le cadre de notre planning scolaire, notre planning s'articule autour de :

1. Les différentes tâches

Tableau 5: Tableau des taches

Tâches	Date de début	Date de fin	Durée (en jours)
Etude du projet	06/12/2025	13/12/2025	08
Analyse	14/12/2025	20/12/2025	07
Conception	21/12/2025	31/12/2025	11
Réalisation	01/01/2026	20/01/2026	20
Déploiement	21/01/2026	28/01/2026	08
Clôture	29/01/2026	31/01/2026	03

2. Diagramme de Gantt

Le diagramme de Gantt couramment utilisé en gestion de projet, est un des outils les plus efficaces pour représenter visuellement l'état d'avancement des différentes activités (tâches) qui constituent un projet.

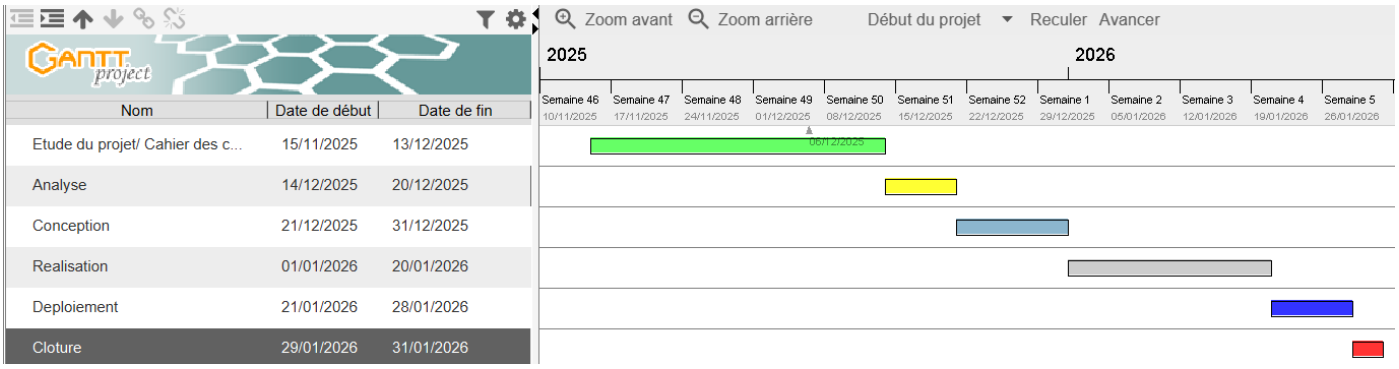

Figure 1: Diagramme de Gantt

VIII. LES CONTRAINTES DU PROJET

1. Les contraintes de coût

Pour mener à terme la réalisation de notre projet, il sera indispensable de dépenser une somme totale de **9 680 000 FCFA** correspondant aux différentes ressources y intervenant.

2. Les contraintes de délais

La réalisation de notre projet devra être effectuée sur une période s'étalant sur 08 semaines de travail allant du **06 Décembre 2025** au **31 Janvier 2025.**

IX. LES LIVRABLES

A la fin de ce projet, en plus du cahier des charges nous devons fournir un :

- Dossier d'analyse
- Dossier de conception
- Dossier de réalisation
- Rapport de déploiement
- Rapport du projet

CONCLUSION

Au terme de notre cahier des charges, il est visible que nous avons une vue globale mais aussi détaillée de tout ce qu'il faut comme ressources pour la réalisation de ce projet. Ces ressources sont d'ordre matérielle, logicielle, et humaine. Toutefois, une estimation du plan financier étant déjà connue, il nous revient maintenant d'approcher la prochaine partie qui est le dossier d'analyse.

PARTIE 2: CAHIER D'ANALYSE

INTRODUCTION

Le projet d’un analyseur avancé de fichier **File Analyzer** vise à simplifier et à rendre accessible l’analyse détaillée des fichiers stockés sur une partition EXT4 sous Debian. Ce cahier d'analyse présente une étude approfondie des besoins fonctionnels et non fonctionnels du système, les méthodes et langages de modélisation utilisés ainsi que les différents diagrammes d'analyse pour mieux comprendre le fonctionnement de l'application.

I. CONTEXTE ET OBJECTIF

1. Contexte

Avec l'évolution des technologies de stockage, les systèmes de fichiers modernes offrent désormais un accès aléatoire, permettant de récupérer ou de modifier rapidement n'importe quelle donnée à partir de son emplacement. Un système de fichiers constitue ainsi un composant essentiel d'un système d'exploitation : il assure l'organisation, le stockage, la gestion et l'accès aux données sur un support tel qu'un disque dur, une clé USB ou une mémoire flash. Il joue également un rôle d'interface entre les applications et le matériel, rendant possibles des opérations comme la création, la lecture, l'écriture ou la suppression de fichiers. Parmi les systèmes de fichiers largement utilisés dans les environnements Linux, **EXT4 (*Extended filesystem-version4*)** occupe une place de référence grâce à sa robustesse, ses performances et ses fonctionnalités avancées, notamment la gestion des extents, la journalisation et la réduction de la fragmentation

2. Objectif

L'objectif principal, qui est le pourquoi de notre projet est **de Faciliter la compréhension et l'analyse approfondie des systèmes de fichiers EXT4** en fournissant un outil unifié permettant d'explorer de manière claire et accessible la structure interne, les métadonnées et l'organisation physique des fichiers sous Debian.

II. PERIMETRE FONCTIONNEL

- Détecter automatiquement le type du fichier analysé (texte, binaire, exécutable, image, PDF, etc.) ;
- Extraire et afficher tous les champs du header du fichier (magic number, sections, métadonnées structurelles).
- Extraire les métadonnées stockées dans l'inode du fichier (permissions, propriétaire, groupe, taille logique et physique) ;
- Récupérer les horodatages complets (atime, mtime, ctime, btime), le nombre de liens physiques et les flags système (immutable, append-only, etc.).
- Identifier et lister l'ensemble des extents utilisés par le fichier ;
- Extraire pour chaque extent son offset logique, sa longueur en blocs et les numéros de blocs physiques correspondants sur le disque.
- Afficher le contenu brut du fichier lorsqu'il s'agit d'un fichier texte ou lisible, tout en indiquant clairement les fichiers binaires non affichables.
- Calculer le taux de fragmentation en analysant le nombre d'extents et leur distribution sur le disque ;

- Fournir une interprétation qualitative du niveau de fragmentation (faible, modéré, élevé).
- Afficher le taux final sous forme de pourcentage.
- Afficher le contenu brut des fichiers texte ou lisibles de manière formatée ;
- De visualiser les informations analysées sous forme de panneaux ou onglets.
- Détecter les fichiers binaires et proposer un affichage hexadécimal ou signaler leur nature non affichable directement.

III. MODELISATION UML

1. Choix de la méthode d'analyse

Au regard de la complexité croissante des systèmes d'information et suite à cette étude comparative, UML s'impose comme le langage le mieux adapté à notre projet. En tant que langage de modélisation unifié, UML permet de spécifier, visualiser, construire et documenter de manière précise l'architecture d'un système. Grâce à ses différents **diagrammes**, UML offre une description à la fois statique, dynamique et fonctionnelle du système. Il s'avère idéal pour :

- **Concevoir et structurer** une architecture logicielle orientée objet, en proposant des formalismes standards et universels ;
- **Modéliser les données** via les diagrammes de classes (utilisés ici pour représenter les entités), palliant ainsi certaines limites des modèles relationnels traditionnels par une approche plus proche du code ;
- **Formaliser les besoins métiers** à travers les diagrammes de cas d'utilisation, permettant de définir clairement les interactions entre les utilisateurs et le futur système.

2. Diagramme d'analyse : Diagramme de cas d'utilisation

a. Diagramme de cas d'utilisation global du système

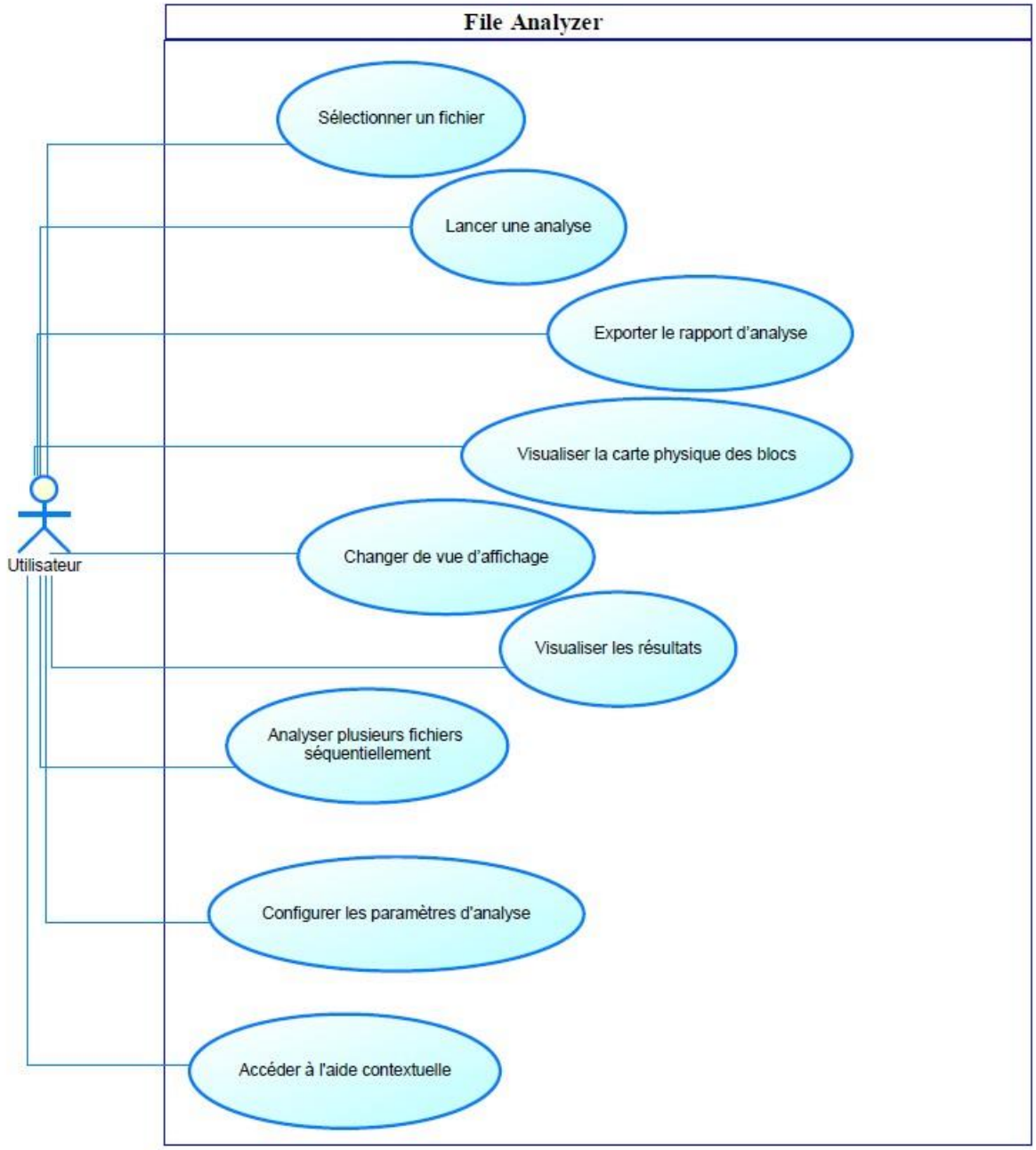

Figure 2: Diagramme des cas d'utilisation

b. Description textuelle des cas d'utilisation

1. CU 1 – Sélectionner un fichier

Tableau 6: CU1-Selectionner un fichier

Titre	Sélectionner un fichier à analyser	
Objectif	Charger un fichier depuis le système de fichiers EXT4 pour préparation à l'analyse.	
Résumé	L'utilisateur navigue dans l'arborescence et choisit un fichier via l'interface graphique.	
Acteur(s)	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Le système fonctionne sous Debian avec partition EXT4.	File Analyzer est lancé, interface chargée.	Le fichier est référencé, prêt pour analyse.
Scénario nominale		
1. Utilisateur clique sur "Parcourir". 2. Fenêtre de sélection s'ouvre. 3. Il navigue vers le fichier. 4. Il sélectionne le fichier → clique "Ouvrir". 5. Le chemin s'affiche, bouton "Analyser" s'active.		
Scénario alternatif		
• Fichier inaccessible → Message "Permission refusée".		

2. CU 2 – Analyser un fichier

Tableau 7: CU2-Analyser un fichier

Titre	Analyser un fichier	
Objectif	Exécuter automatiquement tous les modules d'analyse sur le fichier sélectionné.	
Résumé	L'utilisateur déclenche l'exécution séquentielle des 5 modules définis.	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Un fichier valide est sélectionné.	Fichier chargé, bouton "Analyser" actif.	Tous les résultats sont disponibles dans l'interface.
Scénario nominale		
1. Utilisateur clique "Analyser". 2. Barre de progression apparaît. 3. Modules s'exécutent dans l'ordre : a) Détection type + header, b) Extraction métadonnées, c) Cartographie physique, d) Calcul fragmentation, e) Préparation affichage contenu ; 4. Message "Analyse terminée". 5. Onglets résultats se remplissent.		
Scénario alternatif		
• Échec d'un module → Analyse continue avec mention "Échec partiel". • Fichier trop gros → Proposition d'analyse partielle.		

3. CU 3 – Visualiser les résultats

Tableau 8: CU3-Visualiser les résultats

Titre	**Visualiser les résultats par catégorie**	
Objectif	Consulter les résultats d'analyse organisés par type d'information.	
Résumé	Navigation entre onglets pour voir différents aspects de l'analyse.	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Analyse préalablement effectuée.	Résultats disponibles.	Section choisie affichée à l'écran.
Scénario nominale		
1. Utilisateur clique sur l'onglet "Type & Header". 2. Système affiche magic number, type détecté, structure header. 3. Utilisateur clique sur "Métadonnées". 4. Système affiche permissions, dates, taille, flags. 5. Utilisateur clique sur "Organisation physique". 6. Système affiche liste des extents avec détails.		
Scénario alternatif		
• Onglet vide → Message "Exécutez d'abord l'analyse". • Données volumineuses → Pagination automatique.		

4. CU 4 – Exporter le rapport d'analyse

Tableau 9: CU4-Exporter le rapport d'analyse

Titre	**Exporter le rapport d'analyse**	
Objectif	Sauvegarder les résultats dans un fichier externe.	
Résumé	Génération d'un document contenant tout ou partie des résultats.	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Analyse effectuée.	Résultats disponibles.	Fichier créé sur disque.
Scénario nominale		
1. Utilisateur clique "Exporter". 2. Dialogue s'ouvre : choix format (PDF/HTML/TXT). 3. Choix contenu (complet/résumé/spécifique). 4. Sélection emplacement. 5. Clique "Sauvegarder". 6. Confirmation "Export réussi".		
Scénario alternatif		
• Espace insuffisant → Message d'erreur détaillé. • Échec génération → Proposition format alternatif.		

5. CU 5 – Changer de vue d'affichage

Tableau 10: CU5-Changer de vue d'affichage

Titre	**Changer de vue d'affichage**	
Objectif	Adapter la présentation des résultats selon la préférence.	
Résumé	Basculer entre différentes représentations (texte, tableau, graphique).	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Données à afficher disponibles.	Résultats chargés.	Nouveau mode d'affichage actif.
Scénario nominale		
1. Dans "Organisation physique", utilisateur clique "Vue tableau". 2. Système affiche les extents en tableau. 3. Utilisateur clique "Vue graphique". 4. Système affiche diagramme des blocs. 5. Utilisateur clique "Vue texte". 6. Système affiche description textuelle.		
Scénario alternatif		
• Vue non disponible → Option grisée. • Données incompatibles → Vue par défaut.		

6. CU 6 – Accéder à l'aide contextuelle

Tableau 11: CU6-Accéder à l'aide contextuelle

Titre	**Accéder à l'aide contextuelle**	
Objectif	Obtenir des explications sur les termes techniques.	
Résumé	Consultation de documentation intégrée.	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Documentation incluse.	Application en cours d'exécution.	Explication affichée.
Scénario nominale		
1. Utilisateur survole "extent" avec la souris. 2. Info-bulle apparaît avec définition courte. 3. Il clique sur "?" à côté de "fragmentation". 4. Fenêtre d'aide détaillée s'ouvre. 5. Il peut naviguer dans les sujets liés.		
Scénario alternatif		
• Terme inconnu → "Documentation non disponible". • Demande hors contexte → Menu d'aide général.		

7. CU 7 – Configurer les paramètres d'analyse

Tableau 12:CU7-Configurer les paramètres d'analyse

Titre	**Configurer les paramètres d'analyse**	
Objectif	Personnaliser le comportement de l'outil.	
Résumé	Modification des options et préférences.	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Aucun	Application lancée.	Paramètres sauvegardés.
Scénario nominale		
1. Utilisateur ouvre "Paramètres". 2. Modifie "Taille max d'affichage" à 50 Mo. 3. Active "Toujours afficher l'hexadécimal". 4. Désactive "Vérifier les mises à jour". 5. Clique "Appliquer". 6. Paramètres persistés.		
Scénario alternatif		
• Valeur invalide → Message d'erreur avec contrainte. • Redémarrage nécessaire → Avertissement.		

8. CU 8 – Analyser plusieurs fichiers séquentiellement

Tableau 13: CU8-Analyser plusieurs fichiers séquentiellement

Titre	**Analyser plusieurs fichiers séquentiellement**	
Objectif	Analyser une série de fichiers sans réinitialisation.	
Résumé	Enchaînement d'analyses sur différents fichiers.	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Plusieurs fichiers à analyser.	Application lancée.	Résultats du dernier fichier affichés.
Scénario nominale		
1. Utilisateur analyse le fichier A. 2. Il clique sur "Nouvelle analyse". 3. Sélectionne le fichier B. 4. Clique sur "Analyser". 5. Interface se met à jour avec résultats de B.		
Scénario alternatif		
• Fichier précédent non sauvegardé → Avertissement. • Limite mémoire atteinte → Nettoyage automatique.		

9. CU 9 – Visualiser la carte physique des blocs

Tableau 14: CU-Visualiser la carte physique des blocs

Titre	Visualiser la carte physique des blocs	
Objectif	Voir graphiquement la répartition des données sur le disque.	
Résumé	Représentation visuelle des extents et de la fragmentation.	
Acteur	Utilisateur	
Présupposer	**Précondition**	**Postcondition**
Analyse physique effectuée.	Données d'extents disponibles.	Diagramme affiché.
Scénario nominale		
1. Utilisateur ouvre onglet "Carte physique". 2. Système génère un diagramme coloré. 3. Chaque extent = rectangle coloré. 4. Espaces vides = zones grises. 5. Survol d'un bloc → infos détaillées.		
Scénario alternatif		
• Fichier non fragmenté → Message "Fichier contigu". • Trop d'extents → Vue simplifiée avec zoom.		

IV. LIVRABLES

- Dossier de conception
- Dossier de réalisation
- Rapport de déploiement
- Rapport du projet

CONCLUSION

Le projet File Analyzer présenté dans ce cahier d'analyse ambitionne de fournir une solution complète et pédagogique pour l'analyse approfondie des fichiers sur les systèmes de fichiers EXT4 sous environnement Debian. En combinant des technologies d'analyse bas-niveau, une extraction intelligente des métadonnées et une interface graphique intuitive adaptée aux besoins des utilisateurs, cette solution contribuera à démystifier le fonctionnement interne des systèmes de fichiers et à renforcer les compétences techniques des utilisateurs. Dans la suite nous allons ressortir le dossier de conception de notre projet.

PARTIE 3: CAHIER DE CONCEPTION

INTRODUCTION

Dans la réalisation d'un projet informatique, le dossier de conception est celui qui présente l'architecture du système étudié. L'analyse UML nous permet à travers ses diagrammes structurels de représenter la structure de notre application ainsi que les différents composants matériels et logiciels. Dans cette partie, nous vous présenterons le diagramme de classe ainsi que le diagramme de composant.

I. DIAGRAMMES DE SEQUENCE

1. SEQUENCE: Analyser un fichier

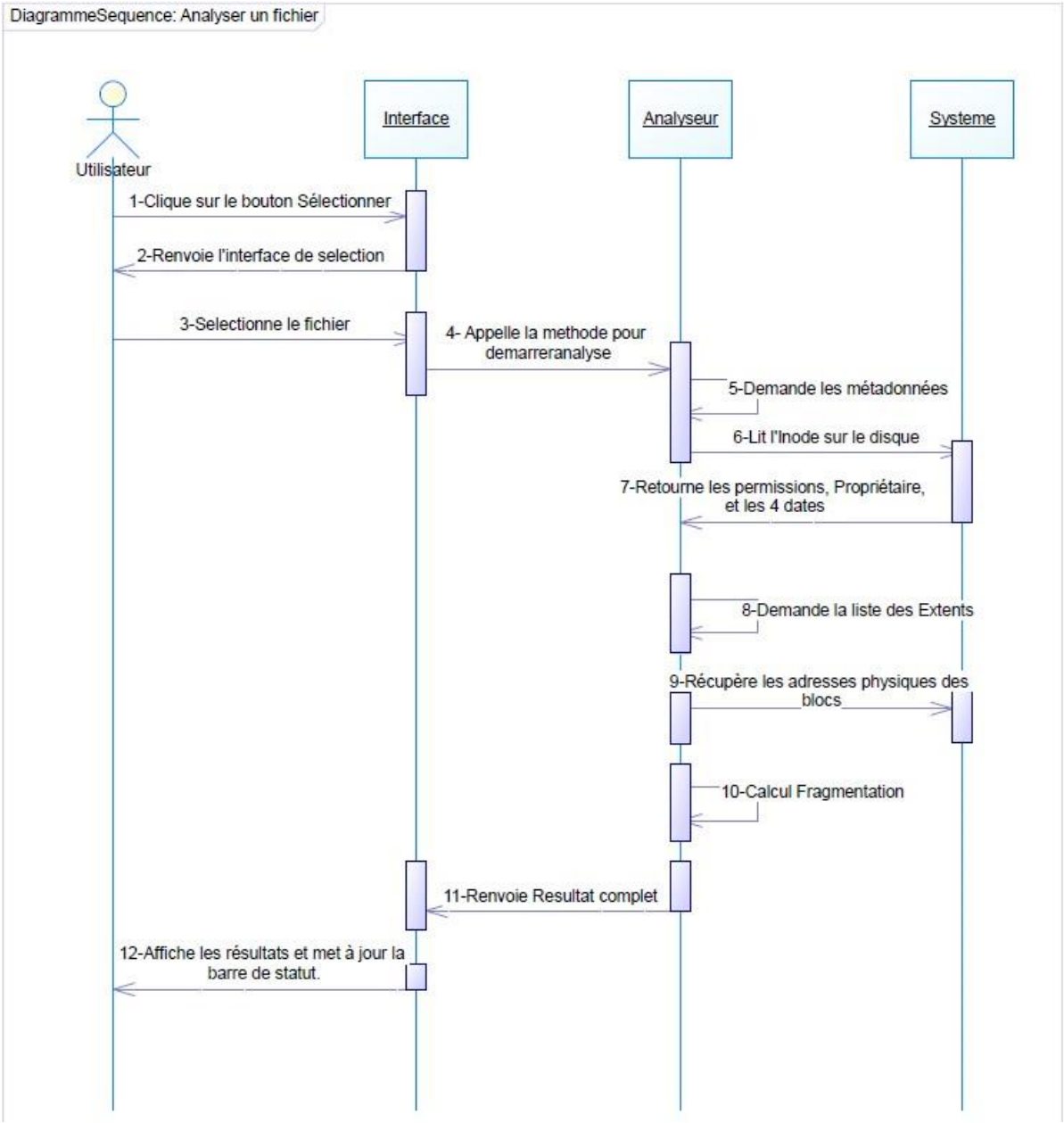

Figure 3: séquence analyser un fichier

2. SEQUENCE : Visualiser la carte physique des blocs

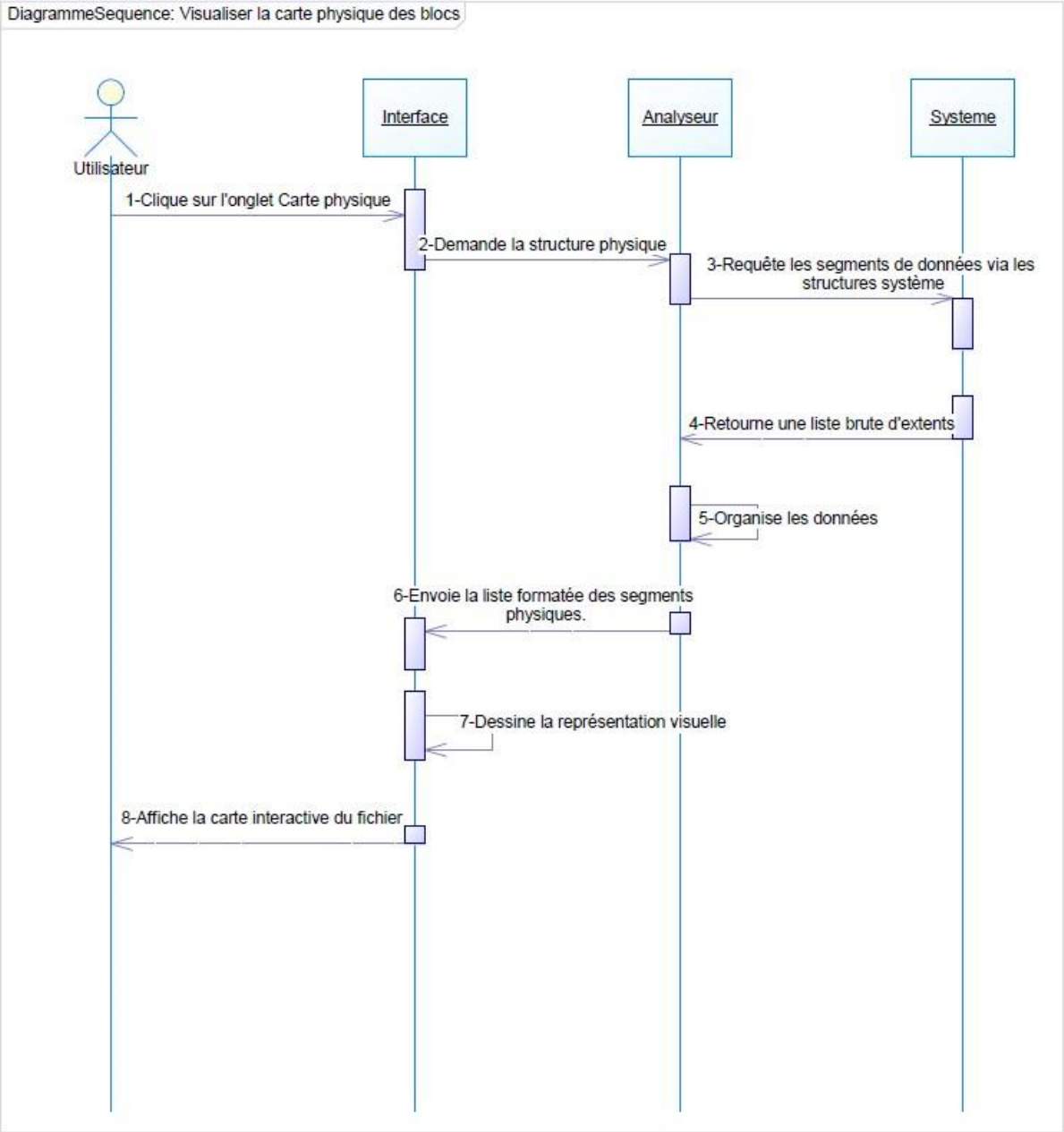

Figure 4: Séquence visualiser la carte physique des blocs

II. DIAGRAMMES DE CLASSE

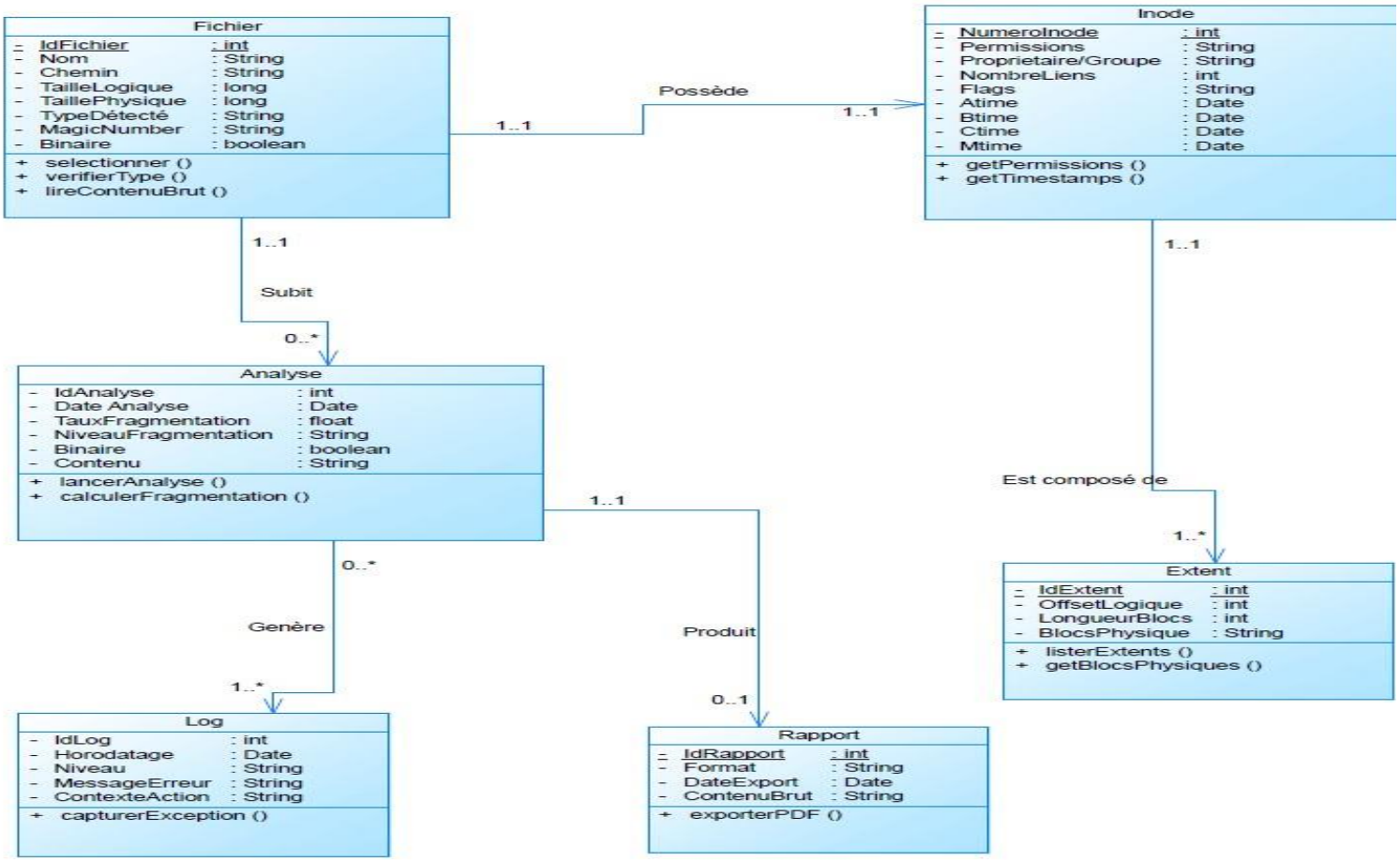

Figure 5: Diagramme de classe

III. LIVRABLES

- Dossier de réalisation
- Rapport de déploiement
- Rapport du projet

CONCLUSION

La phase de conception est une étape primordiale dans la conception logicielle, car elle nous donne une vue globale de la structure de notre application. Après cette étape nous pouvons passer à la réalisation de notre projet.

PARTIE 4: CAHIER DE REALISATION

Introduction

Le dossier de réalisation décrit la manière dont le projet **Linux File Analyzer - EXT4** a été conçu et développé. Il s'adresse aux parties prenantes techniques et permet de comprendre les choix d'architecture, les composants logiciels, et l'implémentation de chaque fonctionnalité.

I. Architecture générale

1. Architecture logicielle

- **Frontend GUI** : Python avec Tkinter, pour l'interface graphique.
- **Module d'analyse** : `analyzer.py` qui utilise :
  - os, stat, pwd, grp pour accéder aux fichiers et métadonnées.
  - subprocess pour interroger le système (ex: lsblk -f).
  - re pour manipuler et parser des chaînes.
  - datetime pour formatage des dates.
- **Export PDF/TXT** : Bibliothèque reportlab (SimpleDocTemplate, Paragraph, Spacer, getSampleStyleSheet, inch) pour générer des rapports.
- **Graphiques** : matplotlib intégré à Tkinter et FigureCanvasTkAgg pour visualiser les cartes physiques et la fragmentation.
- **Système de fichiers cible** : EXT4 (Four-Extended File System).
- **JSON** : json pour stocker et lire les paramètres utilisateur.

2. Organisation des fichiers

```
Linux-File-Analyzer/
├── main.py # Point d'entrée
├── settings.json
├── gui.py # Interface graphique
├── src/ # Modules source
│  ├── analyzer.py # Coordinateur d'analyse
│  ├── file_type.py # Détection type fichier
│  ├── metadata.py # Extraction métadonnées
│  ├── physical_layout.py # Analyse extents
│  ├── fragmentation.py # Calcul fragmentation
│  ├── content_viewer.py # Affichage contenu
│  ├── logger.py # Gestion logs
│  └── utils.py # Utilitaires
├── logs/ # Fichiers de log
├── docs/ # Documentation
```

II. Conception détaillée

1. Interface utilisateur (GUI)

- **Top Frame** :
  - Sélection du fichier (filedialog)
  - Bouton **Analyser**
  - Bouton **Exporter rapport** (PDF/TXT)
  - Indication du système de fichier
- **Notebook / Onglets** :
  - Résumé
  - Métadonnées
  - Organisation physique
  - Fragmentation
  - Carte physique
  - Contenu
- **Widgets** : Text widget, Scrollbar, Boutons pour changer la vue (Texte, Tableau, Graphique), Bouton d'aide contextuelle, info-bulles (ToolTip).
- **Tooltips** : Explications des termes techniques (extent, inode, fragmentation, block).

2. Module d'analyse (analyzer.py)

- Vérification si le fichier est sur EXT4 (subprocess, os.access).
- Retourne un dictionnaire structuré pour alimenter l'interface.
- Extraction des données :
  - Métadonnées (os, stat, pwd, grp, datetime)
  - Organisation physique et fragmentation
  - Contenu (partiel si trop volumineux)

3. Fonctionnalités avancées

- Graphiques dynamiques avec matplotlib pour la **carte physique**.
- Export PDF/TXT avec reportlab.
- Paramètres utilisateur avec settings.json et json.
- Support complet de la visualisation sous plusieurs formats (texte, tableau, graphique).

III. Déroulement de la réalisation

1. Analyse des besoins à partir du cahier des charges.
2. Conception de l'architecture logicielle et choix des bibliothèques.

3. Développement du module analyzer.py avec récupération complète des métadonnées et de la fragmentation.
4. Création de la GUI avec Tkinter, intégration des onglets, Text widgets, Scrollbars et boutons.
5. Ajout de graphiques avec matplotlib.
6. Export PDF/TXT avec reportlab.
7. Gestion des paramètres et stockage JSON.
8. Tests fonctionnels et validation sur différents fichiers EXT4.

IV. Tests réalisés

- Vérification de fichiers contigus et fragmentés.
- Affichage des métadonnées, organisation physique, contenu.
- Export PDF/TXT correct.
- Graphiques pour la carte physique fonctionnels.
- Paramètres utilisateur sauvegardés et chargés correctement.

PARTIE 5: CAHIER DE DEPLOIEMENT

I. INTRODUCTION

Ce document décrit la procédure complète de déploiement de l'application **Linux File Analyzer**, un outil graphique d'analyse approfondie des fichiers sur les partitions EXT4 sous Debian.

II. PRÉREQUIS SYSTÈME

1. Configuration matérielle minimale

Tableau 15: Configuration matérielle minimale

Composant	Spécification minimale	Recommandé
Processeur	Intel Core i3	Intel Core i5
RAM	2 Go	4 Go ou plus
Espace disque	500 Mo	1 Go
Stockage	Partition EXT4	Partition EXT4

2. Configuration logicielle

Tableau 16: Configuration logicielle

Logiciel	Version minimale	Installation
Système d'exploitation	Debian 13.1.0 (Trixie)	lsb_release -a
Python	3.9 ou supérieur	python3 --version
pip	20.0+	pip3 --version
apt	2.0+	apt --version

3. Dépendances système

- Différents paquets requis:

```
sudo apt update
sudo apt install -y \
        python3 \
        python3-tk \
        python3-pip \
        python3-matplotlib \
        python3-reportlab \
        python3-pil \
        file \
        e2fsprogs \
        imagemagick \
```

dpkg-dev

4. Vérification de l'environnement EXT4

- Vérifier que le système de fichiers est EXT4

Lsblk -f | grep ext4

- Vérifier les outils EXT4

which debugfs
which filefrag

III. MÉTHODES D'INSTALLATION

1. Installation rapide (Package .deb)

a. Téléchargement du package

- Depuis le dépôt (si hébergé)

wget https://github.com/OpenSecureFoundation/Linux-File-Analyzer/linux-file-analyzer-1.0.0.deb

- Ou copie depuis une clé USB

cp /media/usb/linux-file-analyzer-1.0.0.deb ~/

b. Installation avec APT (recommandé)

- Installation avec résolution automatique des dépendances

sudo apt install ./linux-file-analyzer-1.0.0.deb

c. Installation avec DPKG

- Installation du package

sudo dpkg -i linux-file-analyzer-1.0.0.deb

- Résolution manuelle des dépendances si nécessaire

sudo apt-get install -f

d. Vérification de l'installation

- Vérifier que le programme est accessible

which file-analyzer

- Doit afficher: /usr/local/bin/file-analyzer
- Vérifier les fichiers installés

ls -la /usr/local/lib/linux-file-analyzer/

ls -la /usr/local/lib/linux-file-analyzer/src/

- Vérifier le fichier desktop

ls -la /usr/share/applications/file-analyzer.desktop

2. Installation depuis les sources

a. Cloner le dépôt

- Via git

git clone https://github.com/belviaine/linux-file-analyzer.git

cd linux-file-analyzer

- Ou décompresser l'archive

unzip linux-file-analyzer.zip

cd linux-file-analyzer

b. Structure du projet

```
linux-file-analyzer/
├── main.py              # Point d'entrée
├── src/                 # Modules source
│   ├── analyzer.py      # Coordinateur d'analyse
│   ├── file_type.py     # Détection type fichier
│   ├── metadata.py      # Extraction métadonnées
│   ├── physical_layout.py  # Analyse extents
│   ├── fragmentation.py    # Calcul fragmentation
│   ├── content_viewer.py   # Affichage contenu
│   ├── gui.py           # Interface graphique
│   ├── logger.py        # Gestion logs
│   └── utils.py         # Utilitaires
├── logs/                # Fichiers de log
├── docs/                # Documentation
├── requirements.txt     # Dépendances Python
└── README.md            # Documentation
```

c. Installation manuelle

- Créer l'environnement virtuel (optionnel)

python3 -m venv venv

source venv/bin/activate

- Installer les dépendances Python

pip install -r requirements.txt

- Tester l'application

python3 main.py

IV. CRÉATION DU PACKAGE .DEB

1. Structure du package

- Créer la structure des dossiers

mkdir -p debian/DEBIAN

mkdir -p debian/usr/local/bin

mkdir -p debian/usr/local/lib/linux-file-analyzer

mkdir -p debian/usr/share/applications

mkdir -p debian/usr/share/icons/hicolor/{16x16,32x32,48x48,64x64,128x128,256x256}/apps

mkdir -p debian/usr/share/doc/linux-file-analyzer

mkdir -p debian/usr/share/man/man1

2. Fichier de contrôle

- Fichier : debian/DEBIAN/control

Package: linux-file-analyzer

Version: 1.0.0

Section: utils

Priority: optional

Architecture: all

Depends: python3 (>= 3.9), python3-tk, python3-matplotlib, python3-reportlab, python3-pil, file, e2fsprogs

Maintainer: SONGMENE LADO Belviane <songmenebelviane.com>

Description: Analyseur avancé de fichiers EXT4

Outil graphique pour analyser en profondeur les fichiers sur les partitions EXT4 sous Debian.

Fonctionnalités:

* Détection automatique du type de fichier

* Extraction des métadonnées (inode, permissions, dates)

* Analyse de l'organisation physique (extents)

* Calcul du taux de fragmentation

* Visualisation graphique de la carte des blocs

* Export PDF/HTML/TXT

3. Script de lancement

- Fichier : debian/usr/local/bin/file-analyzer

```
#!/bin/bash
# Lanceur pour Linux File Analyzer
# Définir le chemin d'installation
INSTALL_DIR="/usr/local/lib/linux-file-analyzer"
# Vérifier les privilèges root
if [ "$EUID" -ne 0 ]; then
    echo "Certaines fonctionnalités nécessitent les droits root (analyse des extents)"
    echo "   Relancez avec 'sudo' pour une analyse complète."
    echo ""
fi
# Lancer l'application (main.py est dans src/)
cd "$INSTALL_DIR/src"
python3 main.py
```

- Rendre exécutable : **chmod +x debian/usr/local/bin/file-analyzer**

4. Fichier .desktop

- Fichier : debian/usr/share/applications/file-analyzer.desktop

```
[Desktop Entry]
Version=1.0
Name=File Analyzer
Name[fr]=Analyseur de Fichiers
Comment=Advanced EXT4 file analyzer
Comment[fr]=Analyseur avancé de fichiers EXT4
Exec=file-analyzer
```

Icon=file-analyzer

Terminal=false

Type=Application

Categories=System;Utility;FileTools;

Keywords=file;analyzer;ext4;fragmentation;metadata;

StartupNotify=true

5. Création de l'icône

- Méthode 1 : Avec ImageMagick (si vous avez un fichier SVG)

```
convert icon.svg -resize 256x256 debian/usr/share/icons/hicolor/256x256/apps/file-analyzer.png
```

- Méthode 2 : Avec Python (script create_icon.py)

```
python3 create_icon.py
#Copier pour toutes les tailles
for size in 16 32 48 64 128 256; do
  cp icon.png debian/usr/share/icons/hicolor/${size}x${size}/apps/file-analyzer.png 2>/dev/null || true
done
```

6. Scripts d'installation

- Fichier : debian/DEBIAN/postinst

```
#!/bin/bash
set -e
Mettre à jour le cache des icônes
if command -v update-icon-caches >/dev/null 2>&1; then
  update-icon-caches /usr/share/icons/hicolor
fi
#Mettre à jour le cache des applications
if command -v update-desktop-database >/dev/null 2>&1; then
  update-desktop-database
fi
#Créer un lien symbolique
```

ln -sf /usr/local/bin/file-analyzer /usr/bin/file-analyzer 2>/dev/null || true

#Créer le dossier de logs s'il n'existe pas

mkdir -p /usr/local/lib/linux-file-analyzer/logs

touch /usr/local/lib/linux-file-analyzer/logs/app.log

chmod 666 /usr/local/lib/linux-file-analyzer/logs/app.log

- Fichier : debian/DEBIAN/prerm

#!/bin/bash

set -e

#Supprimer le lien symbolique

rm -f /usr/bin/file-analyzer

- Rendre exécutables :

chmod +x debian/DEBIAN/postinst

chmod +x debian/DEBIAN/prerm

7. Copie des fichiers sources

- Copier les fichiers Python

cp -v main.py debian/usr/local/lib/linux-file-analyzer/ 2>/dev/null || true

cp -v requirements.txt debian/usr/local/lib/linux-file-analyzer/

cp -v README.md debian/usr/local/lib/linux-file-analyzer/

cp -v LICENSE debian/usr/local/lib/linux-file-analyzer/

- Copier le dossier src

cp -rv src/* debian/usr/local/lib/linux-file-analyzer/src/

- Créer les fichiers d'initialisation

touch debian/usr/local/lib/linux-file-analyzer/src/__init__.py

touch debian/usr/local/lib/linux-file-analyzer/logs/app.log

8. Construction du package

- À la racine du projet

cd ~/ProjetSE/Linux-File-Analyzer-main

- Construire le package

dpkg-deb --build debian linux-file-analyzer-1.0.0.deb

- Vérifier le package

ls -la linux-file-analyzer-1.0.0.deb

dpkg-deb --info linux-file-analyzer-1.0.0.deb

dpkg-deb --contents linux-file-analyzer-1.0.0.deb | head -20

V. DÉPLOIEMENT

1. Installation du package

- Méthode recommandée (avec apt)

sudo apt install ./linux-file-analyzer-1.0.0.deb

- Vérification de l'installation

which file-analyzer

file-analyzer

2. Test de l'application

- Lancement normal

file-analyzer

- Lancement avec droits root (analyse complète)

sudo file-analyzer

3. Test des fonctionnalités

1. Sélection de fichier : Cliquer sur "Parcourir"
2. Analyse : Cliquer sur "Analyser"
3. Vérification des onglets :
    - Résumé (type, taille, métadonnées)
    - Métadonnées (permissions, dates)
    - Organisation physique (extents)
    - Fragmentation (taux, niveau)
    - Carte physique (graphique)

- Contenu (aperçu)

VI. CONFIGURATION

1. Fichier de configuration

Le fichier dont l'emplacement est principalement dans le repertoire : ~/.config/file-analyzer/settings.json

```
{
  "max_display_size_mb": 50,
  "show_hex": true,
  "check_updates": true,
  "theme": "clair"
}
```

2. Fichiers de logs

Emplacement : /usr/local/lib/linux-file-analyzer/logs/app.log

Commande pour voir les logs

tail -f /usr/local/lib/linux-file-analyzer/logs/app.log

VII. DÉPANNAGE

1. Erreurs courantes

Tableau 17: Erreurs courantes

Erreur	Cause	Solution
Permission denied	Droits insuffisants	Lancer avec sudo
No module named ...	Dépendance manquante	pip install -r requirements.txt
main.py not found	Mauvais chemin	Vérifier /usr/local/bin/file-analyzer
EXT4 required	Mauvais FS	Vérifier avec df -T
debugfs not found	e2fsprogs manquant	sudo apt install e2fsprogs

2. Vérifications système

- Vérifier les droits

sudo -v

- Vérifier Python

python3 --version

- Vérifier les dépendances

dpkg -l | grep -E "python3-tk|python3-matplotlib|python3-reportlab|python3-pil"

- Vérifier les logs

cat /usr/local/lib/linux-file-analyzer/logs/app.log

3. Réinstallation

- Désinstaller

sudo apt remove --purge linux-file-analyzer

- Supprimer les fichiers de configuration

rm -rf ~/.config/file-analyzer

sudo rm -rf /usr/local/lib/linux-file-analyzer

- Réinstaller

sudo apt install ./linux-file-analyzer-1.0.0.deb

VIII. DÉSINSTALLATION

1. Via APT

- Désinstaller

sudo apt remove linux-file-analyzer

- Désinstaller complètement (avec configuration)

sudo apt purge linux-file-analyzer

- Supprimer les dépendances non utilisées

sudo apt autoremove

2. Via DPKG

- La commande pour désinstaller

sudo dpkg -r linux-file-analyzer

- La commande pour désinstaller complètement avec les fichiers de configurations

sudo dpkg --purge linux-file-analyzer

3. Nettoyage manuel

En utilisant les commandes suivantes pour supprimer les fichiers résiduels

- **sudo rm -rf /usr/local/lib/linux-file-analyzer**
- **sudo rm -f /usr/local/bin/file-analyzer**
- **sudo rm -f /usr/share/applications/file-analyzer.desktop**
- **sudo rm -rf /usr/share/icons/hicolor/*/apps/file-analyzer.png**
- **rm -rf ~/.config/file-analyzer**

CONCLUSION

Ce cahier de déploiement a été rédigé suite à l'installation et aux tests de l'application Linux File Analyzer v1.0.0 sur Debian 13.1.0 (Trixie).

TABLES DES MATIERES

SOMMAIRE i
LISTE DES TABLEAUX iii
LISTE DES FIGURES iv
PARTIE 1 : CAHIER DES CHARGES 1
INTRODUCTION 2
I. CONTEXTE ET JUSTIFICATION DU PROJET 3
1. Contexte 3
2. Justification 3
II. PROLEMATIQUE 4
III. LES OBJECTIFS DU PROJET 4
1. Objectif global 4
2. Objectifs spécifiques 4
IV. EXPRESSION DES BESOINS 5
1. Besoins Fonctionnels 5
i. Module « Détection et analyse du type de fichier » 5
ii. Module « Extraction des métadonnées système » 5
iii. Module « Analyse de l'organisation physique » 6
iv. Module « Calcul de fragmentation » 6
v. Module « Affichage du contenu » 6
vi. Module « Interface utilisateur graphique (GUI) » 6
vii. Module « Gestion des erreurs et logs » 6
2. Besoins Non Fonctionnels 7
i. Performance 7
ii. Compatibilité 7
iii. Fiabilité 7
iv. Utilisabilité 7
v. Maintenabilité 7
vi. Portabilité 7
V. ACTEURS DU PROJET 8
VI. ESTIMATION DU COUT DU PROJET 8

1. Ressources matérielles 8
2. Ressources logicielles 9
3. Ressources humaines 9
VII. PLANIFICATION DU PROJET 10
1. Les différentes tâches 10
2. Diagramme de Gantt 10
VIII. LES CONTRAINTES DU PROJET 11
1. Les contraintes de coût 11
2. Les contraintes de délais 11
IX. LES LIVRABLES 11
CONCLUSION 12
PARTIE 2: CAHIER D'ANALYSE 13
INTRODUCTION 14
I. CONTEXTE ET OBJECTIF 15
1. Contexte 15
2. Objectif 15
II. PERIMETRE FONCTIONNEL 15
III. MODELISATION UML 16
1. Choix de la méthode d'analyse 16
2. Diagramme d'analyse : Diagramme de cas d'utilisation 17
a. Diagramme de cas d'utilisation global du système 17
b. Description textuelle des cas d'utilisation 17
1. CU 1 – Sélectionner un fichier 18
2. CU 2 – Analyser un fichier 19
3. CU 3 – Visualiser les résultats 20
4. CU 4 – Exporter le rapport d'analyse 21
5. CU 5 – Changer de vue d'affichage 22
6. CU 6 – Accéder à l'aide contextuelle 23
7. CU 7 – Configurer les paramètres d'analyse 24
8. CU 8 – Analyser plusieurs fichiers séquentiellement 25
9. CU 9 – Visualiser la carte physique des blocs 26
IV. LIVRABLES 27

CONCLUSION 28
PARTIE 3: CAHIER DE CONCEPTION 29
INTRODUCTION 30
I. DIAGRAMMES DE SEQUENCE 31
1. SEQUENCE: Analyser un fichier 31
2. SEQUENCE : Visualiser la carte physique des blocs 32
II. DIAGRAMMES DE CLASSE 33
III. LIVRABLES 33
CONCLUSION 34
PARTIE 4: CAHIER DE REALISATION 35
Introduction 36
I. Architecture générale 37
1. Architecture logicielle 37
2. Organisation des fichiers 37
II. Conception détaillée 38
1. Interface utilisateur (GUI) 38
2. Module d'analyse (analyzer.py) 38
3. Fonctionnalités avancées 38
III. Déroulement de la réalisation 38
IV. Tests réalisés 39
PARTIE 5: CAHIER DE DEPLOIEMENT 40
I. INTRODUCTION 41
II. PRÉREQUIS SYSTÈME 41
1. Configuration matérielle minimale 41
2. Configuration logicielle 41
3. Dépendances système 41
4. Vérification de l'environnement EXT4 42
III. MÉTHODES D'INSTALLATION 42
1. Installation rapide (Package .deb) 42
a. Téléchargement du package 42
b. Installation avec APT (recommandé) 42
c. Installation avec DPKG 42

d. Vérification de l'installation 42
2. Installation depuis les sources 43
a. Cloner le dépôt 43
b. Structure du projet 43
c. Installation manuelle 43
IV. CRÉATION DU PACKAGE .DEB 44
1. Structure du package 44
2. Fichier de contrôle 44
3. Script de lancement 45
4. Fichier .desktop 45
5. Création de l'icône 46
6. Scripts d'installation 46
7. Copie des fichiers sources 47
8. Construction du package 47
V. DÉPLOIEMENT 48
1. Installation du package 48
2. Test de l'application 48
3. Test des fonctionnalités 48
VI. CONFIGURATION 49
1. Fichier de configuration 49
2. Fichiers de logs 49
VII. DÉPANNAGE 49
1. Erreurs courantes 49
2. Vérifications système 49
3. Réinstallation 50
VIII. DÉSINSTALLATION 50
1. Via APT 50
2. Via DPKG 50
3. Nettoyage manuel 51
CONCLUSION 51
TABLES DES MATIERES 52